État d'implémentation face à la gouvernance

> ***Statut : constat.*** *Ce document décrit ce qui a été vérifié dans le code, avec ses sources. Les objectifs et les plans sont dans* `cible/`*.*

> *Responsable : Cédric · Version 0.3 du 20/10/2026 · Relevé sur* `main` *après la fusion des PR #14 à #21, #24 et #26 (moteur de remédiation, détection ML, tableau de bord)*
>
> *Les documents de gouvernance ont été écrits avant le code. Ce registre dit, pour chaque hypothèse importante, si le code la* ***confirme****, la* ***contredit*** *ou ne l'a* ***pas encore implémentée****. Il est à mettre à jour après chaque fusion importante et avant les essais (WOR-73).*

1. Ce qui existe au 20/10/2026

Composant	État	Où
API de paiement (Node.js) : sessions, idempotence des paiements, métriques Prometheus, journaux JSON, limitation à 60 requêtes par minute et par client	Implémentée. **Dans le socle, elle garde tout en mémoire** (voir §2)	`app/backend/`, `infra/docker-compose.yml`
Front web et page d'administration	Implémentés	`app/front/`
Base PostgreSQL de l'application	Implémentée, **sans sauvegarde**, utilisée seulement par le déploiement `app/`	`app/infra/docker-compose.yml`
Socle : Vault, PostgreSQL, Prometheus (collecte toutes les 5 s), Loki (rétention 7 jours), Grafana	Implémenté ; l'API y est collectée	`infra/docker-compose.yml`
WAF ModSecurity + OWASP CRS, en mode bloquant, seul point d'entrée publié	Implémenté, l'API est derrière lui. Injections bloquées en 403. Liste d'IP bloquées rechargée à chaud (`nginx -s reload`, 20 s maximum) ; les autres règles demandent un redémarrage	`infra/waf/`
Détection ML : Isolation Forest par famille (`traffic` → déni de service, `injection` → injection), fenêtre de 60 s au pas de 10 s, alerte après 2 fenêtres anormales	Implémentée, en conteneur. **Modèle entraîné sur du trafic normal seulement** : aucun taux ni délai de détection mesuré	`ml/inference/`, `ml/training/report.md`
Moteur de remédiation : reçoit les alertes signées, exécute une liste fermée de playbooks Ansible, vérifie par une transaction de test	Implémenté, **sur l'hôte** (pas en conteneur), avec les droits Docker de l'utilisateur	`services/remediation/`, `scripts/start-remediation.sh`
Playbooks de remédiation : blocage d'IP au WAF, redémarrage, restauration (recréation du conteneur), levée des blocages	Implémentés	`ansible/playbooks/remediation/`
Tableau de bord d'incident (WOR-40), servi par le moteur	Implémenté	`services/dashboard/`
Tableau de bord SOC / SIEM dans Grafana	Implémenté ; aucune règle d'alerte Grafana	`infra/grafana/`
Remise à l'état sain	Implémentée ; **efface base, journaux, métriques et Vault**, lève les blocages	`scripts/reset-known-good.sh`

Composant	État	Où
Export des journaux de Loki vers un dossier de preuves (empreintes SHA-256)	Implémenté, **à lancer à la main** : ni la remise à l'état sain ni le moteur ne l'appellent	`scripts/export-logs.sh`
Durcissement des conteneurs (non-root, `cap_drop`, limites de ressources)	Implémenté	`infra/docker-compose.yml`
Reconnaissance faciale pour la double authentification du back-office (WOR-95)	Implémentée, interne	`services/face-recognition/`
Assistant RAG en lecture seule	Implémenté	`rag/docker-compose.rag.yml`
Scripts d'attaque A et B	Implémentés ; **l'attaque B vise encore une route inexistante** (voir §3)	`attack/`

La chaîne attaque → détection → remédiation → vérification existe maintenant de bout en bout dans le socle. Elle n'a pas encore été chronométrée.

2. Hypothèses de la gouvernance confrontées au code

Point principal : dans le socle, l'API garde ses données en mémoire. Elle n'utilise PostgreSQL que si `STORAGE_DRIVER=postgres` (`app/backend/api/src/main.ts`), variable absente pour `payment-api` dans `infra/docker-compose.yml`. Le playbook de restauration le dit lui-même : « payment-api est en mémoire : le recréer le remet dans l'état du seed » (`restore-service.yml:2`). Conséquences :

- un redémarrage (attaque A) ou une restauration (attaque B) efface **toutes** les transactions, légitimes comprises ;
- le RPO = 0 est impossible, même sans attaque ;
- la « restauration » n'est pas une restauration de données, c'est un retour à l'état de départ.

Hypothèse	Document	Verdict	Constat
Aucune transaction confirmée perdue (RPO = 0)	`cible/rto-rpo.md`	**Contredit**	API en mémoire : chaque redémarrage ou restauration perd les transactions. Aucune sauvegarde
Après une injection : snapshot puis rejeu des transactions légitimes	`cible/rto-rpo.md` §5, `cible/pca-pra.md` S2	**Contredit**	Le moteur recrée le conteneur (`restore-service.yml`) : pas de snapshot, pas de rejeu
Un humain valide la restauration après une injection	`cible/recommandations.md` REC-2, `cible/pca-pra.md` S2	**Contredit**	Restauration automatique sur une seule alerte (`engine.py`)
Deux signaux concordants avant une action destructrice	`cible/pca-pra.md` S4	**Non implémenté**	Une seule alerte ML suffit. Elle combine Isolation Forest et enveloppe en « ou », ce qui n'est pas deux signaux indépendants

Hypothèse	Document	Verdict	Constat
Au plus 3 remédiations en 10 minutes par service	`cible/pca-pra.md` §3	**Contredit**	2 tentatives par incident puis escalade (`config.py:44`, `MAX_ACTION_ROUNDS`), sans limite d'un incident à l'autre
Déni de service : limitation de débit par source, levée après 30 min	`cible/pca-pra.md` S1	**Contredit**	**Blocage complet en 403** de toute IP pesant au moins 20 % du trafic suspect (`ml/inference/service.py:36`). Pas de levée automatique : seule la remise à l'état sain débloque
Reprise vérifiée par une transaction réelle	`cible/rto-rpo.md` §1	**Confirmé**	Connexion, intention de paiement, code, paiement réussi, à travers le WAF (`services/remediation/remediation/probe.py`). Jusqu'à 6 essais espacés de 3 s
Constat d'intégrité avant restauration	`cible/pca-pra.md` S2	**Partiel**	Un simple appel à `/health`, qui répond toujours « ok »
Journaux copiés avant toute restauration	`cible/pca-pra.md` S2, §6	**Partiel**	`scripts/export-logs.sh` existe mais n'est appelé ni par le moteur ni par la remise à l'état sain
Moteur de remédiation protégé (droits limités, ordres authentifiés)	`cible/pca-pra.md` S5, `cible/recommandations.md` REC-4	**En grande partie confirmé**	Alertes signées HMAC avec contrôle de fraîcheur et anti-rejeu, liste fermée de playbooks, IP de l'hôte jamais bloquées, écoute locale. Mais il tourne sur l'hôte avec les droits Docker
Escalade vers un humain	`cible/pca-pra.md` §3	**Partiel**	Seulement après échec (événement `escalation`). Pas d'escalade systématique après une injection
Collecte toutes les 5 s, détection en 20 à 70 s	`cible/rto-rpo.md` §4	**Non mesuré**	Paramètres conformes, mais le modèle n'a jamais été évalué sur des fenêtres d'attaque
Durée de reprise affichée = RTO	`cible/rto-rpo.md` §1	**À corriger à la mesure**	Le moteur compte depuis la réception de l'alerte (T3 → T6), pas depuis le début de l'impact (T1)
Assistant RAG sans accès privilégié	`cible/pca-pra.md` §5	**Confirmé**	Conteneur en lecture seule, sans droits ni accès au moteur Docker
Hôte unique, aucune sauvegarde hors de l'hôte	`cible/nis2-dora.md`, `cible/pca-pra.md` S7	**Confirmé**	Inventaire Ansible local uniquement

Hypothèse	Document	Verdict	Constat
Secrets dans HashiCorp Vault	`cible/ebios-rm.md` BS9	**Partiellement contredit**	Vrai pour le socle et la clé HMAC (fichier hors Git). L'application `app/` garde son mot de passe de base en clair et sa clé OpenAI en variable d'environnement

3. Nouveaux constats à intégrer à l'analyse de risques (WOR-67)

Constat	Risque concerné	Gravité du constat
Restauration automatique qui efface les transactions légitimes, déclenchée par une seule alerte ML	ER6 (remédiation en erreur), ER2 altération	**Élevée** : un faux positif d'injection suffit à perdre toutes les transactions
Blocage complet d'une IP qui pèse 20 % du trafic, sans levée automatique	ER6, R1	Moyenne : un partenaire légitime au fort volume peut être coupé jusqu'à une action manuelle
Moteur de remédiation sur l'hôte avec les droits Docker de l'utilisateur	R3, ER6	Moyenne : le bien support le plus sensible n'est pas isolé ; non mentionné dans `docs/governance/durcissement.md`
Port de la base de l'application (55432) publié sur toutes les interfaces, mot de passe en clair dans le dépôt	R5 exfiltration	**Élevée** : la base est joignable depuis le réseau `RESILIENCE-G6`
L'assistant de discussion de l'application envoie les messages à OpenAI	Fuite de données (ER3), prestataire TIC tiers (DORA art. 28)	Moyenne : données fictives, mais incohérent avec un assistant « local »
Code de vérification des paiements fixe (`123456`) **et renvoyé dans la réponse de l'API** (`demoCode`)	R4 usurpation, R2 fraude	Moyenne en démo, critique en production
Vault avec une seule part de clé, clé de descellement et jeton root dans un fichier de l'hôte	R3, R7	Moyenne : acceptable pour un prototype, à citer en limite
Attaque B sur `/api/pay`, qui n'existe pas (les routes sont en `/v1/…`)	Validité des essais	**Élevée** : l'attaque B ne touche pas l'API ; elle ne prouve que le blocage par le WAF
Pas de solde ni de débit dans le modèle de données (intentions de paiement uniquement)	ER2 altération	Faible : le risque devient « paiements altérés ou dupliqués »

4. Ce qu'il faut avant les essais de mercredi

Par ordre de priorité, pour que la démo prouve ce que la gouvernance affirme :

1. **Brancher l'API du socle sur PostgreSQL** (`STORAGE_DRIVER=postgres`) et faire un `pg_dump` avant chaque essai. Sans cela, chaque remédiation perd les transactions. Infrastructure, Développement.
2. **Évaluer le modèle sur des fenêtres d'attaque** : taux et délai de détection, faux positifs. IA / Data.
3. **Recâbler l'attaque B** sur une route en `/v1/…`. Cybersécurité.
4. **Arbitrer la restauration après injection** : automatique (code actuel) ou validée par un humain (REC-2). Puis aligner le code ou la gouvernance. Gouvernance, Développement.
5. **Appeler `scripts/export-logs.sh`** avant toute restauration et avant la remise à l'état sain. Développement, Infrastructure.
6. **Chronométrer T1 → T6** sur trois essais par attaque (`cible/rto-rpo.md` §6). Tous.

7. **Fermer le port 55432 et passer les secrets de l'application dans Vault.** Cybersécurité.